

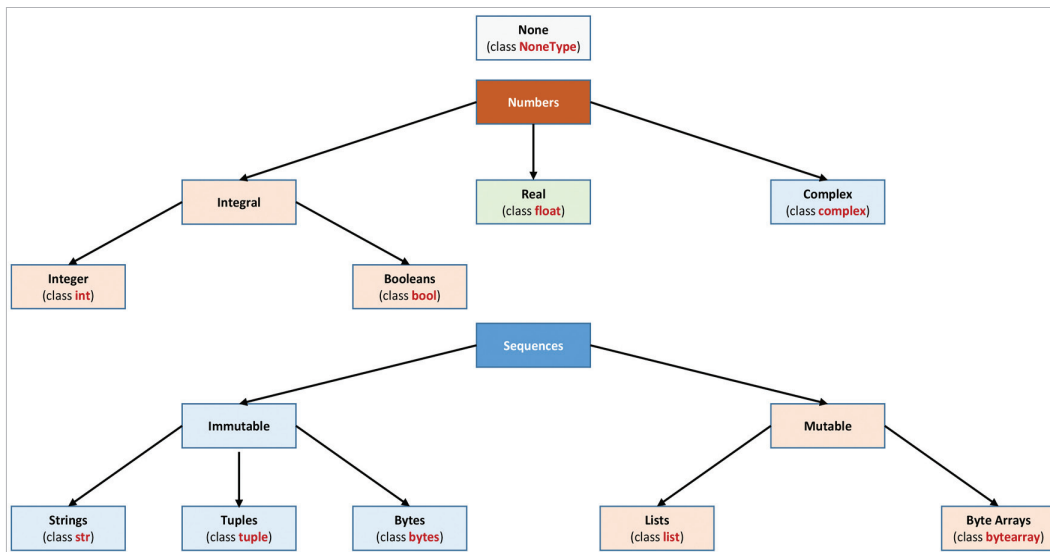


Fig. 4: Python data type hierarchy

According to one survey done by Statista (a German company specialising in market and consumer data), Python is the most popular programming language in the world. Unlike any other language, Python gives the options to build ML programs using its robust library and cross-compilation ability. Moreover, the syntax used in Python is simpler than that of C/C++ or Java.

For example, if a program is written to print the line 'Hello World' using C/C++, Java, and Python, it will be done as follows:

```

// Print "Hello World" using C++ Program
#include <iostream>
int main() {
    std::cout << "Hello World!";
    return 0;
}

// Print "Hello World" using Java
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

# Print "Hello World" using Python
print('Hello, world!')
  
```

As you can see, to print 'Hello World' using C/C++ and Java, 4-5 lines of code are needed, whereas in Python it can be done using a single line of code. The syntax used in Python is like the English language and it's easy to comprehend.

Another special feature of Python

when compared with other object-oriented programming (OOP) languages like C++ and Java is that it is possible to write Python code without making any use of the OOP concept. Python code can be used interpretively. Any Python statement can be interpreted using the interpreter prompt (`>>>`), and can be executed immediately without compiling the whole program. This is just like how the interpreter works in the BASIC programming language, but such a feature is not available in C/C++ or Java. In Python, variables are not required to be declared explicitly, but in C/C++ and Java, the variables must be declared and their type remains static.

For writing various types of code, the ability of the programming language to handle various data types is also important. Python not only supports the primitive data types like Character, Boolean, Integer, and Floating Point like C/C++, Java and R programming languages, but also additional data types like None, Complex Number, Dictionary, and Tuple, which makes it more flexible to implement complex algorithms using Python code. Python also supports various mathematical functions with the help of its libraries, which makes it possible to program different types of code for ML.

Cross-compilation is another great feature of Python and any code written in it can be compiled to the C/C++ programming language. This can be done using CPython, which is a refer-

ence implementation of Python and can compile Python-like code to C/C++.

Another reason for using Python for ML programming is the vast collection of libraries that are designed for the latter. These libraries include Numpy, Scipy, Scikit-learn, Theano, TensorFlow, Keras, PyTorch, Pandas, and Matplotlib. Such libraries make it possible to implement ML algorithms with simplicity and convenience.

The syntax used in Python is simple and any mathematical statements can be expressed with minimum coding. For example, if you need to implement the Naive Bayes algorithm using Python, it can be done using the code:

```

def bayes_algo(p_a, p_b_a, p_b_not_a):
    not_a = 1 - p_a
    p_b = p_b_a * p_a + p_b_not_a * not_a
    p_a_b = (p_b_a * p_a) / p_b
    return p_a_b
  
```

You can see that it's very simple to implement such mathematical formulae using the Python code, without the need to explicitly declare any variable or including any preprocessor/directive like `#include`, in C/C++.

As simple is better than complex, Python can be used for the development of various complex applications with optimised programming code. One empirical study found that Python is more productive than conventional languages, such as C/C++ and Java, for programming problems involving string manipulation or dictionary searches, and the memory consumption is also better than Java. That's why many large organisations like Google, Facebook, Amazon, etc, use Python, which is also helping such companies to grow. **EFY**

This article was first published in March 2021 issue of Open Source For You



Debojit Acharjee is a software engineer/writer